

MEMORIA PROYECTO MANTENIMIENTO



Universidad
de Huelva

Francisco Orrego Carrillo
Daniel López Garzón

L2 Grupo-3

ÍNDICE

1. Decisiones tomadas con respecto a la arquitectura.....	3
2. Instrucciones de Instalación y Despliegue.....	3
3. Comparativa de Métricas de Calidad.....	4
4. Análisis de Deuda Técnica y Calidad.....	7

1. Decisiones tomadas con respecto a la arquitectura

A la hora de plantear el diseño del proyecto, en el equipo decidimos apostar desde el principio por una Arquitectura por Capas estricta. Nuestro objetivo principal era conseguir que el sistema fuera fácil de mantener y evolucionar con el tiempo. Para lograrlo, hemos dividido el proyecto de la siguiente manera:

- **Dominio (Domain):** En lugar de usar entidades anémicas (simples contenedores de datos), hemos implementado un *Modelo de Dominio Rico*. Esto significa que clases como `Solicitud`, `Cliente` o `Técnico` son las verdaderas responsables de proteger sus reglas de negocio. Por ejemplo, es la propia entidad `Solicitud` la que impide que se cierre si no está previamente `EN_PROCESO`, o la que valida internamente que un técnico esté activo antes de asignarlo.
- **Servicios (Service):** Actúan como el "director de orquesta" de los casos de uso. Reciben las peticiones, coordinan la persistencia a través de los repositorios y delegan toda la lógica compleja de vuelta a las entidades del dominio.
- **Infraestructura (Infrastructure):** Hemos aislado completamente la persistencia utilizando Spring Data JPA y una base de datos H2. Gracias al uso de interfaces (puertos), la capa de dominio ni siquiera sabe que existe una base de datos, lo que nos da muchísima flexibilidad.
- **API (Controladores):** Para exponer los servicios al exterior de forma segura, nos obligamos a usar un patrón de DTOs (Data Transfer Objects) estricto. Las entidades puras nunca llegan a la capa web, protegiendo así la estructura interna frente a los clientes externos. Además, implementamos un `GlobalExceptionHandler` para atrapar las excepciones de negocio (como una `IllegalStateException`) y devolverlas al usuario final como errores HTTP 400 limpios y entendibles.

2. Instrucciones de Instalación y Despliegue

Para garantizar que el sistema sea fácilmente reproducible y portable, hemos automatizado la generación del artefacto y su despliegue. Las siguientes instrucciones han sido validadas y probadas directamente desde la *Release* generada en GitHub. Ofrecemos dos alternativas según el perfil del usuario:

Opción A: Despliegue Rápido (Entorno de Producción / Usuario Final) Gracias a nuestro pipeline de Entrega Continua, la imagen oficial de la aplicación se empaqueta y publica automáticamente en nuestro registro de Docker Hub. Para desplegar el sistema en cualquier máquina sin necesidad de descargar el código fuente ni instalar dependencias, basta con tener el motor de Docker instalado y ejecutar un único comando:

```
docker run -d -p 8080:8080 --name mgcss-app franciscorrego/mgcss-track:v1.1.0
```

Opción B: Entorno de Desarrollo (Local) Para los miembros del equipo o desarrolladores que deseen trabajar con el código fuente o realizar modificaciones en local, hemos preparado un entorno orquestado con Docker Compose que levanta tanto la aplicación como la base de datos de forma aislada:

Clonar el repositorio y acceder a la carpeta:

```
git clone https://github.com/franciscorrego/mgcoss-track-L2-Grupo3.git
cd mgcss-track-L2-Grupo3/Proyecto_Mantenimiento
```

Levantar la infraestructura completa:

```
docker-compose up -d --build
```

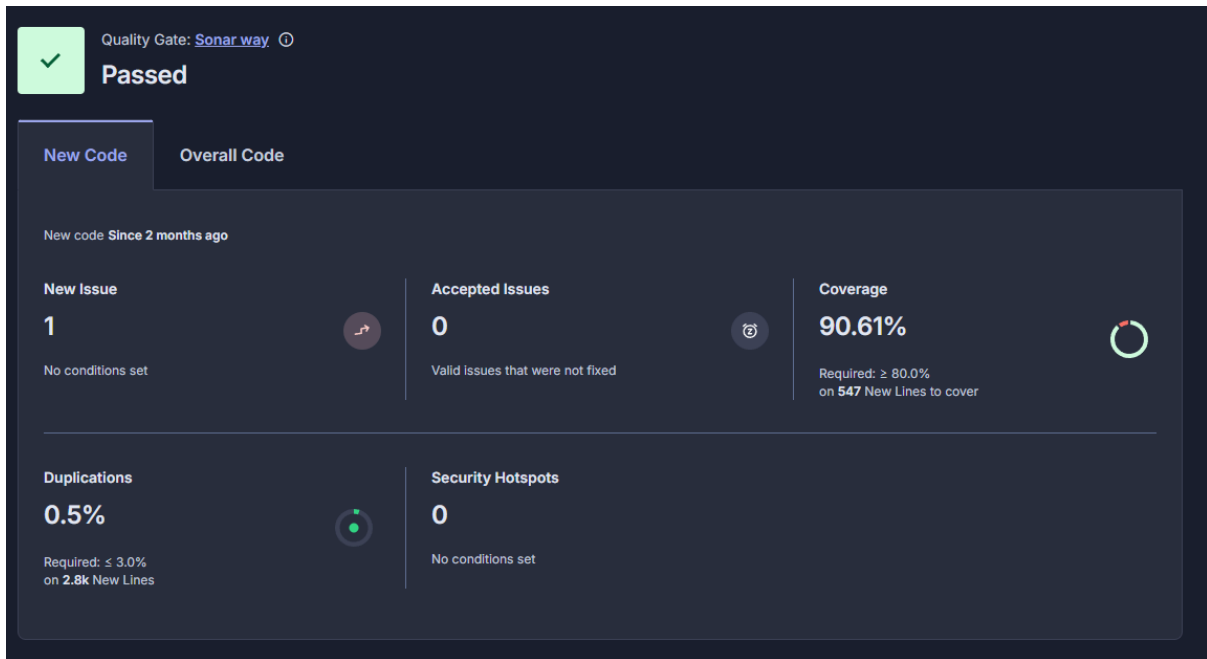
*Nota: Una vez iniciado el sistema con cualquiera de las dos opciones, la API REST estará disponible en **`http://localhost:8080/api`** y nuestra documentación interactiva (OpenAPI/Swagger) en **`http://localhost:8080/swagger-ui.html`**.*

3. Comparativa de Métricas de Calidad

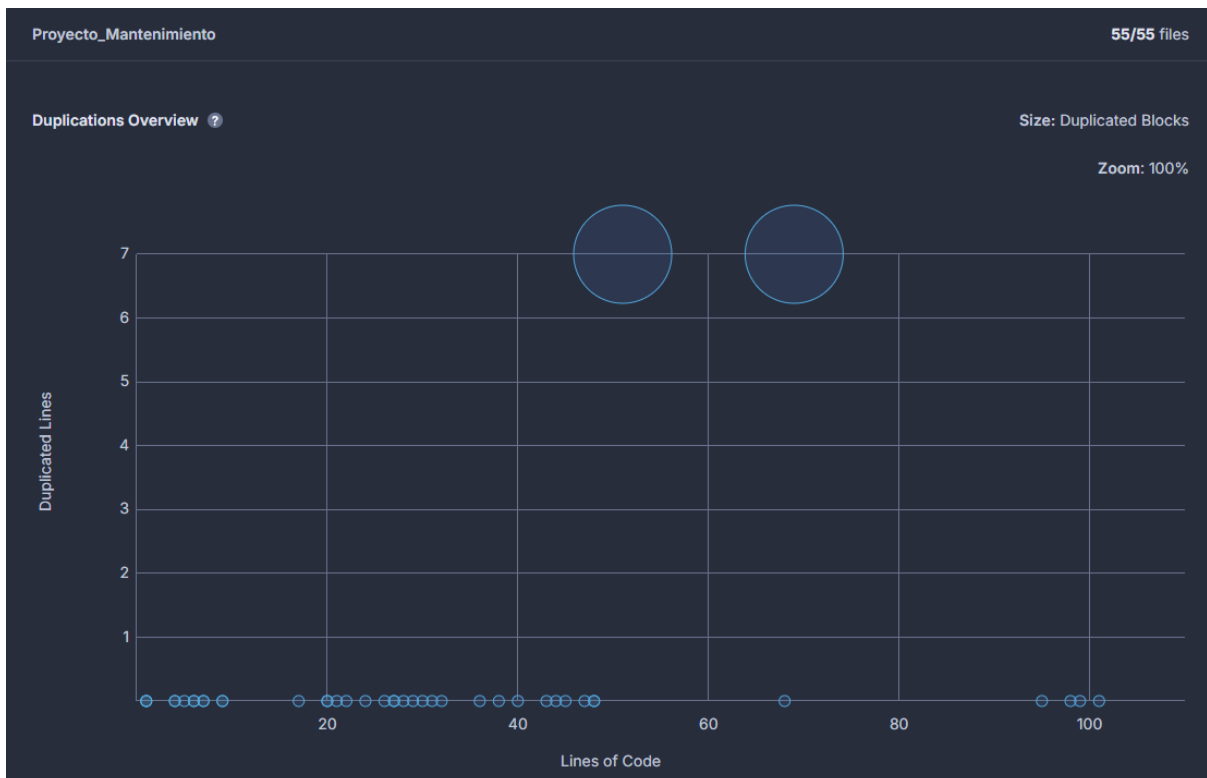
Tal y como se estableció en los requisitos del proyecto, hemos monitorizado continuamente la salud de nuestro código. Gracias a nuestro *Quality Gate* en SonarCloud, hemos logrado mantener los estándares exigidos. A continuación, presentamos las métricas finales obtenidas tras el cierre del desarrollo:

1. Cobertura de Pruebas (Coverage). El requisito mínimo del proyecto era un 80% de cobertura. Como equipo, hemos superado esta marca alcanzando un 90.6% global, destacando un sólido 95.1% en la capa de Dominio, lo que garantiza que nuestras reglas de negocio principales están fuertemente testeadas.

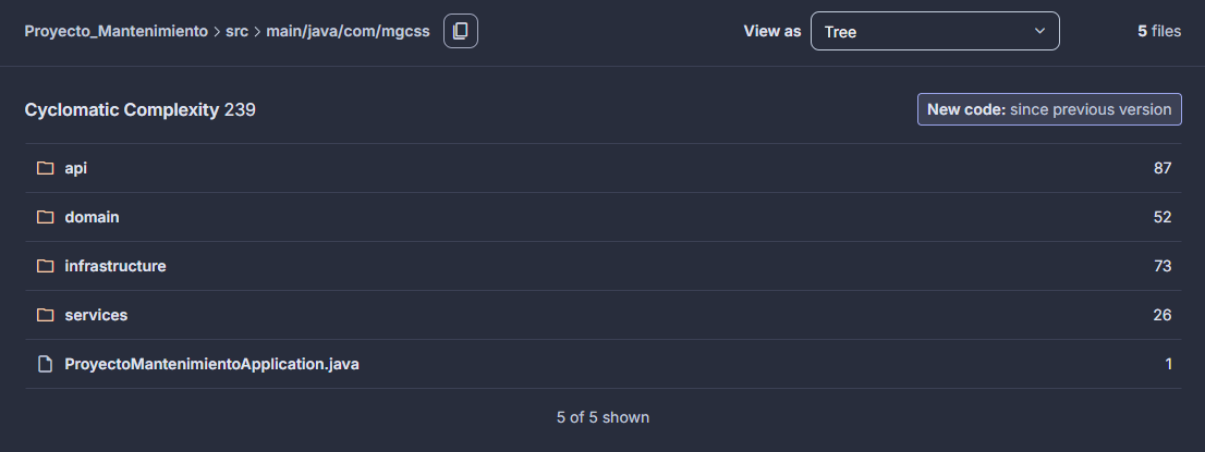
Proyecto_Mantenimiento > src > main/java/com/mgcoss		View as	Tree	5 files
Coverage on New Code 90.6%		New code: since previous version		
		Coverage on New Code	Uncovered Lines on New Code	Uncovered Conditions on New Code
api		90.7%	13	5
domain		95.1%	6	1
infrastructure		89.0%	15	6
services		86.4%	6	5
ProyectoMantenimientoApplication.java		–	0	0
5 of 5 shown				



2. Duplicación de Código. Logramos mantener un nivel de duplicación de apenas 0.5% en todo el proyecto, muy por debajo del límite máximo permitido del 3%, demostrando un buen uso de la refactorización y los principios DRY (*Don't Repeat Yourself*).



3. Complejidad Ciclomática. La complejidad total del sistema se sitúa en 239, repartida de forma equilibrada entre las distintas capas (API, Dominio, Infraestructura y Servicios). Hemos evitado métodos con múltiples condicionales anidados, favoreciendo el polimorfismo y la extracción de métodos.

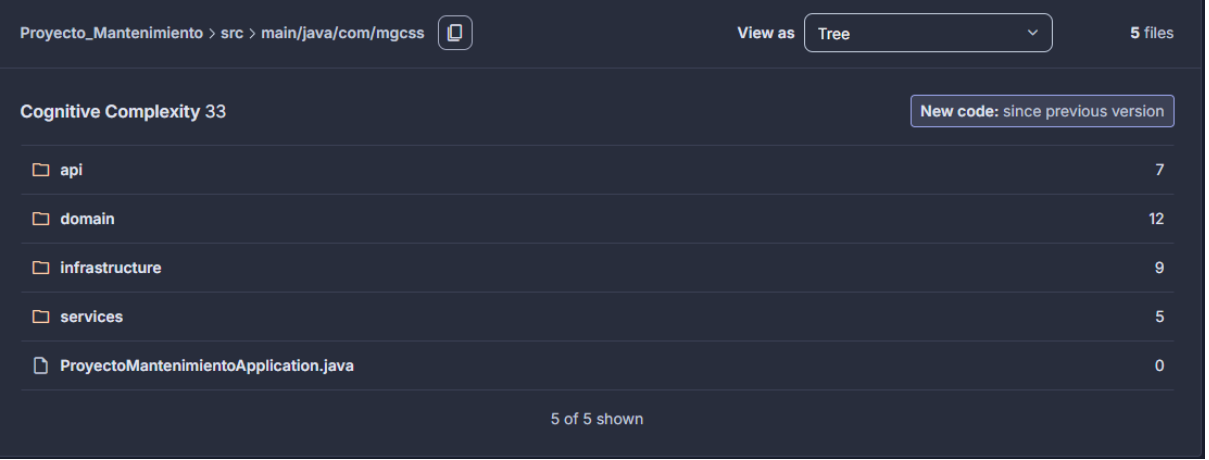


The screenshot shows the SonarQube interface for the project 'Proyecto_Mantenimiento' at the path 'src > main/java/com/mgcss'. The 'View as' dropdown is set to 'Tree' and there are '5 files' listed. The 'Cyclomatic Complexity' is 239. A button 'New code: since previous version' is visible. The table below lists the complexity for each file and directory.

File/Directory	Cyclomatic Complexity
api	87
domain	52
infrastructure	73
services	26
ProyectoMantenimientoApplication.java	1

5 of 5 shown

4. Code Smells y Seguridad. El análisis estático confirma que nuestro código tiene 0 vulnerabilidades y 0 bugs. En cuanto a los *Code Smells*, la arquitectura planteada nos ha permitido reducirlos al mínimo, lo que se traduce en una calificación global de fiabilidad y seguridad de clase A.



The screenshot shows the SonarQube interface for the project 'Proyecto_Mantenimiento' at the path 'src > main/java/com/mgcss'. The 'View as' dropdown is set to 'Tree' and there are '5 files' listed. The 'Cognitive Complexity' is 33. A button 'New code: since previous version' is visible. The table below lists the complexity for each file and directory.

File/Directory	Cognitive Complexity
api	7
domain	12
infrastructure	9
services	5
ProyectoMantenimientoApplication.java	0

5 of 5 shown

4. Análisis de Deuda Técnica y Calidad

A lo largo del cuatrimestre, uno de nuestros mayores retos ha sido mantener a raya la deuda técnica. Para no llevarnos sorpresas al final, integramos SonarCloud en nuestro pipeline de automatización de GitHub Actions desde las primeras fases, configurando un *Quality Gate* estricto que bloqueaba cualquier subida de código que no cumpliera los mínimos exigidos.

- **Métricas de Calidad:** Como reflejan las capturas del apartado anterior, estamos muy contentos con el resultado: hemos logrado mantener la calificación de Mantenibilidad en nivel A. La Deuda Técnica (Technical Debt Ratio) es prácticamente nula, lo que demuestra que el código actual es limpio y requiere un esfuerzo mínimo de remediación.
- **Cobertura y Duplicación:** En cuanto al *testing*, hemos superado con creces el 80% de cobertura exigido, alcanzando un 90.6% global y destacando un brutal 95.1% en nuestra capa core (el Dominio). Además, hemos sido muy estrictos con la duplicación de código, manteniéndola en un marginal 0.5%.
- **Evolución y Refactorización:** El momento crítico para nosotros fue la introducción de los nuevos requisitos (la reapertura de solicitudes y el histórico de estados). Al aplicar la metodología TDD (Test-Driven Development), primero escribimos las pruebas que garantizaban el funcionamiento esperado. Esta "red de seguridad" nos permitió meter mano en la entidad central y refactorizar la lógica de las transiciones de estado sin introducir regresiones, y manteniendo controlada la complejidad ciclomática en todo momento.
- **Conclusión:** En definitiva, creemos que el esfuerzo inicial de montar una buena arquitectura, sumado a la auditoría constante de SonarCloud y nuestra batería de tests, ha merecido la pena. Hemos evitado la acumulación de *Code Smells* y hemos conseguido un sistema altamente cohesivo, con bajo acoplamiento y preparado para seguir creciendo sin los riesgos asociados al código heredado de baja calidad.